

COMSATS University Islamabad

Sahiwal Campus

(Department of Computer Science)

COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE (COAL)

Assignment # 1

Submitted To: Sir Gohar Liaqat

Submitted By: MUHAMMAD HAMZA

Reg #: FA23-BCS-149

Q1-

ANSWER:

(1) **Physical Address:** 12340h

Offset: 0040h

Segment = (Physical Address - Offset) / 10h

Segment = (12340h - 0040h) / 10h

Segment Address = 1230h

(2) **Physical Address:** 12347h

Offset: 0045h

Segment = (12347h - 0045h) / 10h

Segment = 12302h / 10h

Segment Address = 230.2h (Invalid)

Segment address value must be whole number, so it is invalid.

Difficulty:

The problem arises when various combinations of segment and offset results in same physical memory addresses. This is called segment overlapping.

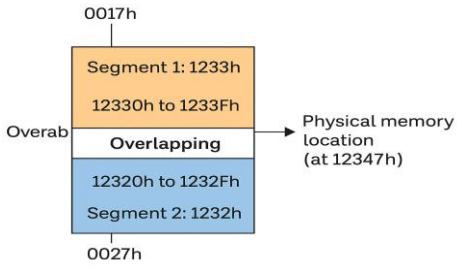
Solution for Difficulty:

Instead of straight calculations, we search combinations:

For example:

1233h:0017h → 12330h + 0017h = 12347h
1232h:0027h → 12320h + 0027h = 12347h

Both are valid. This proves that segment overlapping exists.



Both segments overlap in the memory range from **12330h** to **1233Fh**

Real Life Example:

Consider a warehouse where two managers are responsible for storing goods on shelves. Each manager has their own list where they store items. But sometimes their lists overlap i.e. an item is present on the same shelve.

Pros & Cons:

Pros:

Memory usage is flexible

Memory is efficiently shared between programs

Cons:

It is harder to track and debug memory

Risk of accessing unintended data

Issues Resolved:

Protected Mode:

In this mode, each memory segment has complete knowledge about its size and boundaries. This helps in keeping different parts of memory safe from each other.

Linear Addressing:

Linear addressing is a method which avoids overlapping. Each address points to a specific location which makes memory management easy.

Virtual Memory:

This method provides separate memory spaces to programs. This separation prevents confusion and programs run without interfering with each other.

Q2-

ANSWER:

CPU manages to work upon these IO methods while a program is running as follow:

Interrupt Handling:

When we interact with our computer i.e. (we press a key) the device doesn't wait silently. It sends an **interrupt signal** to the CPU. That signal tells the CPU something happened that needs your attention. The CPU then **stops its current work**, checks what is the input (for example, what key was pressed), processes it, and then **returns back** to what it was doing. This entire process happens in a **fraction of a second**.

Polling:

Poling is a method in which the CPU keeps checking devices after sometime to see if they need anything. This is the same as the Teacher asks from students, "Do you have a question?" even if nobody raises a hand. This method works, but it's not ideal for complex systems, because it is very time consuming, however it is sometimes used in simpler devices or systems.

Direct Memory Access:

For large movements of data (like copying files or playing videos), the CPU allows the hardware to talk directly with memory, skipping the CPU entirely. It is like handling the task by the assistant so the CPU can focus on other things. This is very helpful in streaming or handling files that are too big for constant CPU supervision.

Multitasking and Context Switching:

Modern CPUs are built to **handle more than one thing at a time**. They can switch between different tasks (called threads) very quickly. One thread might be running a program, while another checks for inputs or sends data to screen.

How the CPU Manages I/O During Program Execution:

When a program is running, the CPU plays an important role in handling input and output (I/O) operations.

1. The CPU begins by executing instructions and manages how the movement of data occurs between its internal registers and the system's memory.

2. If an I/O operation is needed, for example, reading from a keyboard, the CPU can respond in several ways. It might continuously check the device's status (polling), react to a signal from the device (interrupts), or delegate the task to a Direct Memory Access (DMA) controller to handle data transfer more efficiently.
3. In the case of an interrupt, the CPU temporarily pauses its current task and runs a special routine, known as an Interrupt Service Routine (ISR), to deal with the I/O request.
4. Some systems also allow the CPU to use dedicated instructions, like IN and OUT, to send and receive data directly to and from I/O devices.
5. After handling the I/O, the CPU returns to its main task, ensuring the program is running smoothly while I/O tasks are either managed in the background or by system resources.

Q3-

ANSWER:

Cache memory works like a human brain, it tries to keep the most useful and frequently needed information ready before it is asked for use.

Locality of Data Access:

This concept explains that programs don't access data randomly. They follow predictable patterns:

- **Temporal Locality:**
  If some data or instruction is accessed once, it is likely to be used again soon.
  Example: If your program checks a variable now, it may check it again shortly.
- **Spatial Locality:**
  If one memory location is accessed, the program is likely to access nearby memory locations too.
  Example: If you traverse through an array, the CPU will access one element after another, those which are stored near each other in memory.

Locality of Assignment Reference:

This is related to how variables are assigned and used in a program. Typically, variables are declared and used just after the declaration.

Key Factors in Cache Management:

Caching systems use specific criteria to determine which data should be kept readily available for quick access. Two important considerations include:

1. **Access Frequency**: Data which is often requested tends to stay in the cache longer, because repeated use of the data suggests it's important for performance.
2. **Recent Usage**: Data that has been accessed recently is also prioritized, it was assumed that it may be needed again soon.

How This Applies to All 3 Types of Cache

L1 Cache (Level 1 - Fastest, Smallest)

- Closest to the CPU
- Stores **most recently used instructions and data**
- Depends heavily on **temporal locality**
- Example: Instructions inside a tight loop or a frequently used variable

L2 Cache (Level 2 - Slower, Bigger)

- Stores data that might not be needed instantly but was used recently
- Acts as a backup if L1 cache does not have it
- Uses both **temporal** and **spatial** locality
- Example: Other parts of a function that was just called

L3 Cache (Level 3 - Shared, Larger)

- Shared between CPU cores
- Holds common data that is used across different tasks or threads
- Mostly depends on **spatial locality** and broader memory access patterns
- Example: Frequently accessed global variables, shared libraries